Complete Crypto Wallet Security & Development Course

Beginner to Advanced Professional Level

Purpose of this Course This course is designed to teach you:

- How crypto wallets work
- How to securely create, manage, and recover wallets
- Wallet architecture and blockchain fundamentals
- Seed phrases, BIP standards, HD wallets, derivation paths
- Safe and legitimate wallet recovery techniques
- Wallet development and scripting

- Security best practices
- Professional troubleshooting methods
- Ethical and legal guidelines

This course is strictly for legal, authorized, and ethical use.

MODULE 1 — INTRODUCTION TO CRYPTO WALLETS

What Is a Crypto Wallet?

A crypto wallet is a tool that stores private keys and allows users to:

- Send crypto
- Receive crypto
- Sign blockchain transactions
- Access decentralized applications

A wallet does NOT actually store coins. Coins

remain on the blockchain. The wallet stores the cryptographic keys needed to control blockchain addresses.

Types of Wallets

1. Hot Wallets

Connected to the internet.
Examples:

- MetaMask
- Trust Wallet
- Phantom

Advantages:

- Easy to use
- Fast transactions
- Good for daily use

Disadvantages:

- More exposed to hacks

2. Cold Wallets

Offline wallets.

Examples:

- Ledger
- Trezor
- Air-gapped wallets

Advantages:

- Extremely secure

Disadvantages:

- Less convenient

Custodial vs Non-Custodial Wallets

Custodial

A third party controls your keys. Example:

- Centralized exchanges

Non-Custodial

You control your private keys. Example:

- MetaMask
- Trust Wallet

Rule: "Not your keys, not your coins."

MODULE 2 — BLOCKCHAIN FUNDAMENTALS

Core Concepts

Public Key

Can be shared publicly. Used to generate addresses.

Private Key

Secret cryptographic key. Controls wallet funds.

Address

Public destination for receiving crypto.

Hashing

One-way cryptographic process.
Common algorithms:

- SHA256
- Keccak256
- RIPEMD160

Elliptic Curve Cryptography

Bitcoin and Ethereum use ECC.

Bitcoin:

- secp256k1

Used for:

- Signing transactions
- Generating keypairs

MODULE 3 — SEED PHRASES & BIP STANDARDS

BIP39 Explained

BIP39 defines mnemonic seed phrases.
Example:

- 12-word seed phrase
- 24-word seed phrase

Purpose: Human-readable backup for wallet recovery.

Official BIP39 Wordlist

Official source:

url BIP39 GitHub Repositoryhttps://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki

Wordlist:

url Official English Wordlisthttps://github.com/bitcoin/bips/blob/master/bip-0039/english.txt

Entropy Basics

Entropy creates randomness.

Examples:

- 128-bit entropy → 12 words
- 256-bit entropy → 24 words

BIP32 — HD Wallets

Hierarchical Deterministic Wallets.

Features:

- Infinite addresses from one seed
- Organized derivation paths

BIP44 — Derivation Paths

Standard structure:

m / purpose' / coin_type' / account' / change / address_index

Bitcoin:

m/44'/0'/0'/0/0

Ethereum:

m/44'/60'/0'/0/0

MODULE 4 — IAN COLEMAN TOOL

What Is Ian Coleman?

A widely used offline BIP39 tool.

Functions:

Generate seed phrases

Convert mnemonics

Generate addresses

Check derivation paths

Official Ian Coleman Tool

urlIan Coleman BIP39 Toolhttps://iancoleman.io/bip39/

GitHub:

urlIan Coleman GitHub Repositoryhttps://github.com/iancoleman/bip39

SAFE USAGE GUIDE

Recommended:

Download the tool

Disconnect internet

Run locally offline

Never expose real seeds online

Offline Setup

Download Repository

```bash
git clone https://github.com/iancoleman/bip39.git
```

Open Offline

Open:

```bash
src/index.html
```

inside browser while offline.

MODULE 5 — INSTALLING DEVELOPMENT ENVIRONMENT

Windows Setup

Install:

Python

urlPython Official Website https://www.python.org/downloads/

Git

urlGit Official Website https://git-scm.com/downloads

VS Code

urlVisual Studio Code https://code.visualstudio.com/

```bash
sudo apt update
sudo apt install python3 python3-pip
git
```

Termux Setup (Android)

Install Termux

urlTermux GitHub Releases https://github.com/termux/termux-app/releases

Update Packages

```bash
pkg update && pkg upgrade
```

Install Python & Git

```bash
pkg install python git
```

MODULE 6 — PYTHON BASICS FOR WALLET DEVELOPMENT

Variables

```python
name = "Bitcoin"
price = 95000
```

Functions

```python
def greet():
    print("Hello")
```

Loops

```python
for i in range(5):
    print(i)
```

MODULE 7 — WORKING WITH WEB3 & BITCOIN LIBRARIES

Install Web3

```bash
pip install web3
```

Official Docs:

urlWeb3.py Documentationhttps://web3py.readthedocs.io/

Install Bip Utils

```bash
pip install bip-utils
```

Official:

urlbip-utils GitHubhttps://github.com/ebellocchia/bip_utils

Install bitcoinlib

```bash
pip install bitcoinlib
```

Official:

urlbitcoinlib Documentationhttps://bitcoinlib.readthedocs.io/

MODULE 8 — GENERATING A WALLET USING PYTHON

Ethereum Wallet Generator

python

```
from eth_account import Account

acct = Account.create()

print("Address:", acct.address)
print("Private Key:", acct.key.hex())
```

Generate BIP39 Mnemonic

python

```
from mnemonic import Mnemonic

mnemo = Mnemonic("english")
words =
```

```
mnemo.generate(strength=128)

print(words)
```

Install:

```bash
pip install mnemonic
```

MODULE 9 — HD WALLET DERIVATION

Example Using bip-utils

```python
from bip_utils import
Bip39SeedGenerator
from bip_utils import Bip44
from bip_utils import Bip44Coins
```

```
mnemonic = "abandon abandon
abandon abandon abandon abandon
abandon abandon abandon abandon
abandon about"

seed_bytes =
Bip39SeedGenerator(mnemonic).Gener
ate()

wallet = Bip44.FromSeed(seed_bytes,
Bip44Coins.ETHEREUM)

account =
wallet.Purpose().Coin().Account(0).Chan
ge(False).AddressIndex(0)

print(account.PublicKey().ToAddress())
```

MODULE 10 — ETHEREUM RPC & BLOCKCHAIN INTERACTION

Using Infura

urlInfura Official Website https://www.infura.io/

Connect to Ethereum

```python
from web3 import Web3

rpc = "https://mainnet.infura.io/v3/YOUR_API_KEY"

w3 = Web3(Web3.HTTPProvider(rpc))

print(w3.is_connected())
```

Check Balance

```python
address = "0x0000000000000000000000000000000000000000"

balance = w3.eth.get_balance(address)

print(w3.from_wei(balance, 'ether'))
```

MODULE 11 — TRANSACTION SIGNING

Ethereum Transaction Signing

```python
from web3 import Web3

private_key = "YOUR_PRIVATE_KEY'

account =
```

```
w3.eth.account.from_key(private_key)

transaction = {
    'to':
'0x0000000000000000000000000000000000000000',
    'value': w3.to_wei(0.001, 'ether'),
    'gas': 21000,
    'gasPrice': w3.to_wei('20', 'gwei'),
    'nonce':
w3.eth.get_transaction_count(account.address)
}

signed_txn =
w3.eth.account.sign_transaction(transaction, private_key)

print(signed_txn.rawTransaction.hex())
```

MODULE 12 — BITCOIN WALLET BASICS

Generate Bitcoin Wallet

```python
from bitcoinlib.wallets import Wallet

wallet = Wallet.create('MyWallet')

key = wallet.get_key()

print(key.address)
```

MODULE 13 — SECURE BACKUP PRACTICES

Best Practices

DO:

- Store seeds offline
- Use metal backups

Use hardware wallets

Verify addresses carefully

DON'T:

Screenshot seeds

Store seeds in cloud notes

Share private keys

Enter seeds on random websites

MODULE 14 — PROFESSIONAL WALLET RECOVERY PRINCIPLES

Scope of Legitimate Wallet Recovery

This course ONLY covers:

Recovery of wallets you personally own

Recovery for clients with written authorization

Recovery of corrupted backups
Recovery of forgotten derivation paths
Recovery of damaged wallet files
Recovery of incomplete seed phrases where ownership is verified

This course does NOT cover:

Unauthorized wallet access
Attacking wallets that do not belong to you
Malware deployment
Credential theft
Phishing
Exploit development

REAL-WORLD WALLET RECOVERY CASES

Case 1 — Wrong Derivation Path

Symptoms:

- Correct seed phrase
- Wrong addresses appearing
- Zero balance shown

Cause: Different wallets use different derivation paths.

Examples:

Bitcoin Legacy:

m/44'/0'/0'/0/0

Bitcoin SegWit:

m/49'/0'/0'/0/0

Bitcoin Native SegWit:

m/84'/0'/0'/0/0

Ethereum:

m/44'/60'/0'/0/0

Case 2 — Missing Passphrase

Symptoms:

- Seed phrase appears valid

- Wallet opens but funds missing

Cause: BIP39 passphrase enabled.
Important: A BIP39 passphrase creates an entirely different wallet.

Case 3 — Wrong Word Order

Symptoms:

- Invalid checksum
- Recovery failure

Solution Workflow:

1. Verify each word
2. Verify spelling
3. Verify order
4. Check checksum
5. Test permutations carefully

Case 4 — Damaged Backup

Examples:

- Water damaged paper
- Incomplete metal backup
- Missing characters
- Corrupted encrypted backup files

PROFESSIONAL RECOVERY WORKFLOW

Step 1 — Gather Wallet Information

Collect:

- Wallet type
- Blockchain used
- Wallet software name
- Approximate creation year
- Known addresses
- Transaction history
- Backup type

Device type

Step 2 — Identify Standards Used

Check:

BIP39
BIP32
BIP44
Electrum seed
Legacy private key format

Step 3 — Verify Seed Integrity

Use:

Offline BIP39 tools
Checksum validation
Wordlist verification

Step 4 — Test Derivation Paths

Most recovery failures come from:

- Wrong derivation paths
- Wrong account indexes
- Wrong address formats

Step 5 — Address Verification

Compare generated addresses against:

- Exchange withdrawal history
- Old screenshots
- Blockchain explorers
- Transaction records

MODULE 15 — IAN COLEMAN PROFESSIONAL GUIDE

Official Ian Coleman Sources

Main Tool:

[Ian Coleman BIP39 Tool]

GitHub Repository:

[Ian Coleman GitHub Repository]

BIP39 Reference Documentation:

[Ian Coleman Documentation]

HOW PROFESSIONALS USE IAN COLEMAN

Primary Uses:

- Seed verification
- Address derivation
- Derivation path discovery
- Wallet compatibility testing
- Address recovery verification
- Extended key analysis

OFFLINE SECURITY PROCEDURE

Recommended Process:

1. Download repository
2. Verify hashes/signatures
3. Disconnect internet
4. Open standalone HTML locally
5. Work completely offline
6. Never paste active seeds online

Community recommendations strongly emphasize offline and air-gapped usage. (reddit.com)

DOWNLOADING IAN COLEMAN LOCALLY

```bash
git clone https://github.com/iancoleman/bip39.git
```

Or download ZIP from GitHub.

RUNNING OFFLINE

Open:

```bash
bip39-standalone.html
```

with internet disconnected.

The tool supports:

- BIP32
- BIP39
- BIP44
- BIP49
- BIP84
- Multiple address types
- Multiple coins

(iancoleman.co)

PROFESSIONAL DERIVATION PATH TESTING

Example Workflow:

1. Enter mnemonic offline
2. Select coin type
3. Test BIP44
4. Test BIP49
5. Test BIP84
6. Compare generated addresses
7. Match against known transaction history

COMMON DERIVATION PATHS

Bitcoin Legacy:

m/44'/0'/0'/0

Bitcoin SegWit:

m/49'/0'/0'/0

Bitcoin Native SegWit:

m/84'/0'/0'/0

Ethereum:

m/44'/60'/0'/0

BTCRecover documentation provides examples of commonly used derivation paths. (docs.btcrecover.org)

MODULE 16 — BTCRECOVER PROFESSIONAL USAGE

Official BTCRecover Resources

GitHub:

BTCRecover GitHub Repository

Documentation:

BTCRecover Documentation

Derivation Path Notes:

BTCRecover Derivation Path Guide

WHAT BTCRECOVER DOES

BTCRecover assists with:

- Password recovery
- Seed phrase typo recovery
- Derivation path scanning
- Wallet file analysis
- Partial mnemonic recovery

Only use on wallets you legally own or are authorized to recover.

PROFESSIONAL SAFETY RULES

Always:

- Use official repositories
- Verify downloads
- Work offline
- Use air-gapped environments
- Avoid unofficial forks

Security discussions in the crypto community

warn about fake or modified copies of recovery tools. (reddit.com)

INSTALLATION

```bash
git clone https://github.com/gurnec/btcrecover.git

cd btcrecover

pip install -r requirements.txt
```

SAFE EXAMPLE WORKFLOW

Example Scenario:

- One recovery word may contain a typo
- Wallet ownership already verified
- Known receiving address available

Workflow:

1. Verify mnemonic length
2. Verify checksum
3. Check similar BIP39 words
4. Test derivation paths
5. Compare generated addresses

COMMON RECOVERY ERRORS

Wrong Coin Type

Example:

> Using Bitcoin path for Ethereum wallet

Wrong Account Index

Example:
m/44'/60'/0'/0
vs

m/44'/60'/1'/0

Wrong Address Type

Examples:

- Legacy
- SegWit
- Native SegWit
- Taproot

MODULE 17 — PYTHON RECOVERY UTILITIES

Install Required Libraries

```bash
pip install mnemonic bip-utils web3
```

BIP39 WORD VALIDATION SCRIPT

python

```
from mnemonic import Mnemonic

mnemo = Mnemonic("english")

phrase = input("Enter mnemonic: ")

if mnemo.check(phrase):
    print("Valid mnemonic")
else:
    print("Invalid mnemonic")
```

GENERATE ETHEREUM ADDRESS FROM SEED

python

```
from bip_utils import
Bip39SeedGenerator
from bip_utils import Bip44
```

```
from bip_utils import Bip44Coins

mnemonic = input("Enter mnemonic: ")

seed =
Bip39SeedGenerator(mnemonic).Gener
ate()

wallet = Bip44.FromSeed(seed,
Bip44Coins.ETHEREUM)

account =
wallet.Purpose().Coin().Account(0).Chan
ge(False).AddressIndex(0)

print(account.PublicKey().ToAddress())
```

DERIVATION PATH TESTING SCRIPT

python

```
paths = [
    "m/44'/0'/0'/0",
```

```
    "m/49'/0'/0'/0",
    "m/84'/0'/0'/0"
]

for path in paths:
    print("Testing:", path)
```

MODULE 18 — PROFESSIONAL RECOVERY ENVIRONMENT

Recommended Setup

Offline Laptop

Used only for recovery work.

Linux Environment

Recommended distributions:

- Ubuntu
- Debian

- Tails

External Storage

Use encrypted USB drives.

Air-Gapped Operation

Disconnect all networking during seed handling.

FORENSIC DOCUMENTATION PRACTICE

Professionals document:

- Wallet type
- Tested derivation paths
- Verified addresses
- Recovery attempts
- Software versions
- Chain

Allowed use cases:

- Recovering your own wallet
- Client-authorized recovery
- Forgotten derivation paths
- Missing address indexes
- Corrupted backups

Common Recovery Problems

Wrong Derivation Path

Examples:

- m/44'/0'/0'/0/0
- m/49'/0'/0'/0/0
- m/84'/0'/0'/0/0

Seed Verification Workflow

1. Verify checksum

2. Check word spelling
3. Test derivation paths
4. Scan addresses
5. Verify balances

MODULE 15 — BTCRECOVER OVERVIEW

Purpose

BTCRecover helps recover wallets when partial information is known.

Official:

urlBTCRecover GitHub Repositoryhttps://github.com/gurnec/btcrecover

Documentation:

urlBTCRecover Documentationhttps://btcrecover.readthedocs.io/

Ethical Rules

Use ONLY when:

- You own the wallet
- You have explicit authorization

Never use against unauthorized wallets.

Install BTCRecover

```bash
git clone https://github.com/gurnec/
btcrecover.git

cd btcrecover

pip install -r requirements.txt
```

MODULE 16 — COMMON WALLET

Areas to Check

Browser Extensions

- MetaMask vaults
- Browser profiles

Mobile Backups

- Encrypted backups
- App data

Hardware Wallet Records

- Seed cards
- Recovery sheets

MODULE 17 — AIR-GAPPED SECURITY

What Is Air-Gapping?

Using completely offline systems.

Advantages:

Protection from remote attacks

Recommended Workflow

1. Offline signing device
2. Online broadcast device
3. QR code transfers

MODULE 18 — MULTI-SIGNATURE WALLETS

What Is Multisig?

Requires multiple approvals.

Example:

2-of-3 signatures

Used by:

Companies
DAOs
Treasury systems

Popular Multisig Solutions

Safe (formerly Gnosis Safe)

url Safe Official Website https://safe.global/

MODULE 19 — HARDWARE WALLETS

Ledger

url Ledger Official Website https://

www.ledger.com/

Trezor

url Trezor Official Website https://trezor.io/

MODULE 20 — ADVANCED SECURITY

Security Checklist

Use:

- Hardware wallets
- 2FA
- Passphrases
- Offline backups

- Dedicated crypto devices

Avoid:

- Public Wi-Fi
- Pirated software
- Unknown browser extensions
- Fake wallet apps

MODULE 21 — IDENTIFYING SCAMS

Common Scams

Fake Airdrops

Fake Wallet Apps

Seed Phrase Phishing

Clipboard Malware

Fake Support Agents

Verification Techniques

Always:

- Verify domains
- Bookmark official sites
- Cross-check URLs
- Verify signatures

MODULE 22 — BUILDING A SIMPLE WALLET APPLICATION

Simple CLI Wallet

python

```python
from eth_account import Account

class Wallet:
    def create_wallet(self):
        acct = Account.create()

        print("Address:", acct.address)
        print("Private Key:", acct.key.hex())

wallet = Wallet()
wallet.create_wallet()
```

MODULE 23 — DATABASE STORAGE BASICS

SQLite Example

python

```python
import sqlite3

conn = sqlite3.connect('wallets.db')

cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS wallets (
    id INTEGER PRIMARY KEY,
    address TEXT,
    private_key TEXT
)
''')

conn.commit()
conn.close()
```

MODULE 24 — ENCRYPTION

BASICS

Fernet Encryption

bash

```
pip install cryptography
```

Example

python

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

cipher = Fernet(key)

encrypted = cipher.encrypt(b"secret")

print(encrypted)
```

MODULE 25 — PROFESSIONAL WORKFLOW

Recommended Learning Path

Beginner

- Wallet basics
- Blockchain fundamentals
- Seed phrases

Intermediate

- Python scripting
- Web3 interaction
- HD wallets

Advanced

Wallet development

Air-gapped systems

Security auditing

Enterprise multisig systems

MODULE 26 — USEFUL RESOURCES

Bitcoin Developer Docs

urlBitcoin Developer Documentationhttps://developer.bitcoin.org/

Ethereum Developer Docs

urlEthereum Developer Portalhttps://ethereum.org/en/developers/

MetaMask Docs

urlMetaMask Developer Docshttps://docs.metamask.io/

Solidity Docs

urlSolidity Official Docshttps://docs.soliditylang.org/

Chainlist RPC Directory

urlChainlist Official Websitehttps://chainlist.org/

MODULE 27 — ETHICS & LEGALITY

Important Rules

Never:

- Attempt unauthorized wallet access

Use phishing methods

Steal credentials

Deploy malware

Use brute-force attacks against systems

Always:

Get written authorization

Maintain client privacy

Work legally and ethically

FINAL RECOMMENDATIONS

Practice Safely

Use:

Testnets

Demo wallets

Small balances

Offline environments

Suggested Practice Projects

1. Build a wallet generator
2. Create a transaction signer
3. Build a portfolio tracker
4. Create a multisig demo
5. Build an offline signing workflow
6. Create a wallet backup checker

END OF COURSE